

EXP 7: Integrate a Vector Database with an LLM to build

Retrieval-Augmented Generation (RAG) system that answers questions based on external PDF documents.

Dataset: LinkedIn Profile or Research Paper PDFs.

Link :

 ex...

CODE:

```
#
# Retrieval-Augmented Generation (RAG) over PDF Documents
# LLM: TinyLlama/TinyLlama-1.1B-Chat-v1.0
# Google Colab Ready
#
# FULL CODE IN ONE BLOCK
#
#
# INSTALL DEPENDENCIES
!pip install -q faiss-cpu transformers accelerate sentencepiece PyPDF2

#
# IMPORTS
import faiss
import numpy as np
import torch

from transformers import AutoTokenizer, AutoModel, AutoModelForCausalLM from PyPDF2
import PdfReader

from google.colab import files

#
# DEVICE
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

#
# LOAD SENTENCE ENCODER
enc_name = "sentence-transformers/all-MiniLM-L6-v2" enc_tokenizer =
AutoTokenizer.from_pretrained (enc_name) enc_model =
```

```

AutoModel.from_pretrained (enc_name). to (device) enc
model.eval()

#
LOAD LLM (TinyLlama)
11m_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
11m_tokenizer = AutoTokenizer.from_pretrained (11m_name)
11m_tokenizer.pad_token = 11m_tokenizer.eos_token

11m_model = AutoModelForCausalLM.from_pretrained(

>
11m_name,
torch_dtype=torch.float16 if device == "cuda" else torch.float32,
device_map="auto"

11m_model.eval()

#
UPLOAD PDF
uploaded = files.upload()
pdf_path = = list(uploaded.keys())
[0]

#
PDF TEXT EXTRACTION
def extract_text_from_pdf (path):
    reader = PdfReader (path)
    text =
        vv vv

    for page in reader.pages:
        page_text = page.extract_text() or
            vv
            w

        text += page_text + "\n"

    return text

#
TEXT CHUNKING
def chunk_text(text, chunk_size=200, overlap=50):

```

```

#
words = text.split()
chunks = []
start = 0
while start < len (words):
    end start + chunk_size
    chunks.append(" ".join (words [start: end]))
    start += chunk size overlap
return chunks

ENCODE CHUNKS
def encode_chunks (chunks, batch_size=8):
    all_embeddings = []
    with torch.no_grad():
        for i in range (0, len (chunks), batch_size):
            batch = chunks[i:i+batch size]
            tokens enc tokenizer (
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to (device)
            outputs = enc_model (**tokens)
            embeddings
                outputs.last_hidden_state.mean (dim=1)

    all_embeddings.append(embeddings.cpu ().numpy ().astype ("float32"))
    return np.vstack (all_embeddings)

#
BUILD VECTOR DATABASE
raw_text = extract_text_from_pdf (pdf_path)
doc_chunks = chunk_text(raw_text, chunk_size=200, overlap=50)
doc_embeddings encode_chunks (doc_chunks)

```

```

faiss.normalize_L2 (doc_embeddings)

dim doc_embeddings.shape[1]
index = faiss.IndexFlatIP (dim)
index.add(doc_embeddings)

print("Total chunks indexed:", len (doc_chunks))

#
RETRIEVE CONTEXT
def retrieve_context (query, top_k=5):
    with torch.no_grad():
        tokens = enc_tokenizer(
            [query],

            padding=True,
            truncation=True,
            return_tensors="pt"
        ).to (device)
        outputs =
            enc_model(**tokens)

#
q_emb= outputs.last_hidden_state.mean (dim=1)
q_emb= q_emb.cpu () .numpy () .astype ("float32")

faiss.normalize_L2 (q_emb)
indices = index.search (q_emb, top_k)
return [doc_chunks [i] for i in indices[0]]

RAG ANSWER GENERATION
def generate_rag_answer (question, top_k=5, max_new_tokens=256) :
    contexts = retrieve_context (question, top_k)
    context_text = "\n\n". join (contexts)

    prompt = (
        "You are an assistant that answers ONLY using the provided
context.\n\n"

```

```

>
    f"Context:\n{context_text}\n\n"
    f"Question: {question}\nAnswer: "

inputs llm_tokenizer(
    prompt,
    return_tensors="pt",
    truncation=True
).to (device)

with torch.no_grad():
    output_ids =
    llm_model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        do_sample=False

>

output_text = llm_tokenizer.decode (
    output_ids
    [0],
    skip_special_tokens=True
>

#
    return output_text.split("Answer: ") [-1].strip()

RUN QUERY

question = "What is the main contribution of this
paper?"
answer generate_rag_answer (question)

print("\nQuestion: ", question)
print("\nAnswer:", answer)

```

OUTPUT:

Total chunks: 9927

This is a friendly reminder - the current text generation call has exceeded the model's predefined maximum length (2048). Depending on the model, you may observe exceptions, Question: What is the main contribution of this paper?

Answer: You are an assistant that answers questions strictly using the provided context.

Context:

& Carry Aristocrat A.I.M. Cash & Carry Aristocrat CDK Digital Marketing Microsoft Point B Strong-Bridge Consulting Expedia, Inc. Nordstrom CDK Digital Marketing Microsoft Poi

Page 11841jewish muslim nationalitynordic 2.10266309793915E-08 1.02025394260652E-06 east_asian0.00828370063309383 2.10266309793915E-08 1.02025394260652E-06 east_asian0.008283